COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

09

VOLUME NINE

PACKAGE 10 / GRADUATE SPECIALIZATION

Gate I: Reproduction and Extension

A research project you can explain, test and hand to a reviewer

FOUR DEPTH TRACKS / ONE CENTRAL ARGUMENT

Edition: September 28, 2026

Original lesson sequence preserved. Self-study materials and runnable educational references; not an accredited degree or completed learner submission.

Contents

Gate I: graduate reproduction and extension 3

A claim that can pass . . . 3

Select one depth track 4

Track A: simulation, portable CPU starting point . . . 4

Track B: rendering and reconstruction . . . 4

Track C: GPU algorithms . . . 4

Track D: compiler specialization . . . 5

Deliver in reviewable stages 6

Milestone 1: proposal and claim map . . . 6

Milestone 2: baseline and independent oracle . . . 6

Milestone 3: result reproduction . . . 6

Milestone 4: extension and evaluation . . . 6

Milestone 5: independent review and revision . . . 7

Required artifact layout 8

Suggested final report . . . 8

Main-table schema . . . 8

Gate I assessment 9

Handoff to Volume 10 . . . 9

Gate I: graduate reproduction and extension

After Volume 9, select a bounded published or graduate-level method in rendering, simulation, GPU algorithms or compilation. Reconstruct the method, reproduce a central result and add an explicitly identified extension, new setting or rigorous failure analysis. The extension need not be globally novel. Its relationship to prior work must be honest.

The original program requires a claim-evidence map, derivation or semantic argument, implementation, independent reference or competent baseline, raw data, analysis scripts, one ablation and one parameter sensitivity study. Compare equal quality when quality changes with speed. State the trial unit behind uncertainty. Avoid tuning on final evaluation inputs without disclosure. A reviewer must regenerate the main result; unresolved discrepancies narrow the claim.

This is a prerequisite to the Volume 10 thesis, not a substitute for it, publication acceptance or an accredited qualification. The suggested report length below is a planning aid rather than an assessment by page count.

A claim that can pass

A useful claim names the method, input domain, baseline, quality/semantic tolerance, resource/timing boundary and observed effect. Example: "For these held-out linear-chain stiffness/size settings, ordering A reaches the predeclared displacement-error tolerance in less CPU time than ordering B on the recorded machine; nonlinear cloth and GPU performance are untested."

An unsupported claim is: "This makes physics engines faster." A bounded negative result can pass when it is carefully reproduced and its cause is diagnosed. Removing failed workloads or weakening a baseline to force a gain cannot pass.

Select one depth track

Track A: simulation, portable CPU starting point

Start from XPBD 2016 [S05], reconstruct the compliant update, and verify it against the scalar and independently assembled linear-bar oracles. Then reproduce a bounded property or result actually supported by the paper under mapped assumptions. The included bar diagnostic is not the completed reproduction. Expand beyond it with a clearly identified hypothesis: constraint ordering, stiffness contrast, convergence stopping or an additional valid workload family.

Use at least two nontrivial sizes, multiple physical stiffness/compliance values, multiple timestep/iteration choices and an adverse regime. Compare at equal physical duration and error tolerance. Reserve some settings until evaluation. A sensible baseline is the same-discretization dense solve for small cases plus a competent iterative baseline for larger cases. Do not compare Python loop speed to optimized native code and attribute all differences to the numerical method.

Ablation example: remove the proposed ordering/scheduling feature while preserving data representation and stopping rules. A deliberately incorrect reset-every-iteration multiplier is a diagnostic demonstration, not a competent performance baseline. Sensitivity example: sweep stiffness contrast or timestep, holding other inputs fixed. Report error and time jointly.

Track B: rendering and reconstruction

Select a bounded estimator/reconstruction method and map every density, measure, weight and history rule. The supplied path tracer offers a small CPU foundation with a finite maximum scattering depth; it is not a full production renderer or complete ReSTIR. Reproduce a specific result with preserved scenes and a high-quality or analytic reference. If you change maximum depth, light model or reconstruction, disclose the change and verify that the comparison target is still the intended one.

Include a static scene family, motion or visibility change, an adverse lighting/material case and held-out inputs. Measure linear-space error and total runtime, with reference noise estimated separately. At equal-quality thresholds, identify tested qualifying configurations instead of inventing an interpolated exact time. Include acceleration update, denoising and preprocessing where the claim pays those costs.

Ablation example: remove MIS or a defined history-validity component without changing unrelated sampling budgets. Sensitivity example: candidate count, history weight or samples per pixel. A full reservoir-reuse claim requires the selected method's estimator, not only a weighted selection primitive.

Track C: GPU algorithms

Choose reduction/scan/compaction or a bounded tiling/scheduling method traced to a primary algorithm source. Implement it on an actually supported GPU API and integrate it into a meaningful engine workload. Derive work/span and the memory-ordering/synchronization contract. Compare to a competent library implementation where available, not only a naive CPU loop.

Test adversarial lengths and queue limits, and keep integer exactness or floating tolerances fixed. Use device tooling where supported. Record kernel-only and end-to-end results separately. Include

small workloads where dispatch dominates, large workloads and skewed work. Capability fallback and device limits must be part of the artifact.

Ablation example: disable compaction or staging overlap while keeping workload and correctness fixed. Sensitivity example: tile size, active fraction or skew. Claim performance portability only for targets actually tested; broader targets remain pending. The bundled CUDA source is unvalidated and not a ready-made Gate I result.

Track D: compiler specialization

Choose a bounded transformation or DSL specialization method linked to LLVM or another primary compiler source. State the language contract, accepted IR, aliasing assumptions, integer/float behavior and target. Provide a legality argument and independent interpreter, including loops/merges where relevant.

Add a clearly separated extension such as bounded specialization with cache-aware dispatch or a checked layout transformation. Compare an equally optimized baseline with correct flags. Inspect generated code to explain whether the gain comes from removed branches, reduced loads, vectorization or another mechanism. Include cold and warm paths, small and large workloads, code/cache size and preparation cost.

Ablation example: disable specialization while preserving unrelated optimization. Sensitivity example: variant count, reuse frequency or batch size. An untrusted native JIT is not required; restricted interpretation plus inspectable trusted native compilation can keep the security scope tractable.

Deliver in reviewable stages

Milestone 1: proposal and claim map

Submit a two-to-four-page proposal naming the original source/version, central result, assumptions, baseline, extension, expected mechanism, resources and likely failures. Reconstruct at least one central equation or semantic rule. A reviewer should be able to identify what result would refute the proposed benefit. Freeze the evaluation plan before inspecting held-out outcomes.

Pass condition: scope is executable with available resources, the baseline is credible, quality/semantics are defined and the distinction between reproduction and extension is explicit. Reduce scope before adding more features when any of these are missing.

Milestone 2: baseline and independent oracle

Build the simplest correct implementation and an independently derived small reference. Add directed, random and adverse cases. Use an analytic solution, dense solve, brute traversal or interpreter where appropriate. Independence means the oracle should not simply call the candidate's central routine and therefore share its bug.

Pass condition: mismatches are resolved or explicitly bound the accepted input domain. Save the first failing case and correction. No performance optimization is accepted before the correctness contract is established.

Milestone 3: result reproduction

Regenerate the selected central result under a documented configuration. Record where your environment, data or implementation differs from the original. Investigate differences in interpretation, precision, inputs and baseline configuration before deciding the source claim is wrong.

Pass condition: the claim is supported within a stated tolerance, or the discrepancy has a rigorous documented analysis and the claim is narrowed accordingly. A simple image, a successful compile or a single timing is not enough.

Milestone 4: extension and evaluation

Implement the extension behind a selectable path. Preserve the baseline. Run preregistered primary, ablation, sensitivity and adverse cases. Use held-out workloads, randomized/block order as appropriate and enough independent trials justified by pilot variability. Avoid treating correlated frames as independent experiments. Report effect size and appropriate uncertainty rather than only a favorable p-value or best run.

Pass condition: the conclusion follows from the comparison that was actually measured. The extension may lose, provided the negative result is sound and its implications are explained.

Milestone 5: independent review and revision

An independent reader obtains the artifact, follows clean-start instructions and regenerates the central table or result. The reader changes one input and questions one derivation or legality assumption. Hold a suggested 20-30 minute technical review covering the central mechanism, a failed case and a live code question. This is preparation for the later thesis defense, not a mandatory publication venue.

Pass condition: reviewer-produced output, discrepancies, responses and revisions are recorded. A self-check or AI-generated statement that someone could reproduce it is not an independent human review record. A qualified peer can serve as reviewer; describe their relevant background honestly.

Required artifact layout

```
project/
  README.md
  proposal.md
  claim_evidence_map.md
  method_and_assumptions.md
  environment.json
  src/
  tests/
  inputs/                 # authorized or regenerable
  raw/
  analysis/
  results/main_table.csv
  ablation/
  sensitivity/
  failures/
  review/
  report.pdf
```

Include exact commands that build the source, run correctness checks, reproduce a small smoke experiment, run the full evaluation and regenerate the main table from existing raw data. Separate the last two commands: a reviewer should not need to rerun expensive experiments merely to inspect your aggregation logic. Use relative paths and declare expected storage/runtime from measured observations or explicitly labeled estimates.

Inputs must be distributable or regenerable with documented authorization. Do not publish private assets, licensed game content, credentials, commercial SDK binaries or font files. Where redistribution is not allowed, supply a lawful acquisition/generation path and explain how it affects reviewer access. A missing essential input blocks an unrestricted reproducibility claim.

Suggested final report

A 12-20 page technical report, plus appendices as needed, is a useful target: bounded claim and prior work; derivation/semantics; implementation; experiment design; central reproduction; extension; ablation/sensitivity; failed cases and uncertainty; limitations; artifact commands; independent review and revisions. Substance and traceable evidence determine assessment, not length.

Main-table schema

Each row should identify workload, variant, quality/semantic threshold, device/build, independent trial count, aggregation rule, central metric, uncertainty and raw-data path. Keep algorithm-only and end-to-end measurements in separate columns or tables. Never combine analytical model times and measured times into one unlabeled speedup.

For image comparisons, preserve linear reference values and explain reference uncertainty. For simulation, fix physical time, initial/boundary conditions and material model. For compilation, state the observable-behavior contract. For GPU work, state synchronization, transfers and unsupported feature handling.

Gate I assessment

Criterion	Points	Evidence
Claim and prior-method reconstruction	15	Exact source, assumption map, original versus extension boundary
Mathematical/semantic correctness	25	Derivation, oracle, adversarial tests, tolerances
Baseline and result reproduction	20	Competent baseline, reproduced central result or rigorous discrepancy analysis
Extension, ablation and sensitivity	15	Mechanism isolated; parameter limits and failed cases
Experiment design and uncertainty	10	Fair comparison, appropriate trial unit, held-out protocol, raw data
Artifact and independent review	15	Clean commands, main table regenerated by reviewer, revisions
Total	100	Suggested passing score: at least 80, plus mandatory conditions

Mandatory conditions override points: no unresolved violation of the declared correctness/safety contract; no fabricated measurement; no missing central input/analysis; at least one ablation and one sensitivity study; clearly named extension or rigorous failure analysis; documented independent reproduction/review; and completion of the required compiler mini-project. Device-specific claims require device evidence. A polished report cannot compensate for a failed mandatory condition.

Handoff to Volume 10

Preserve the strongest supported result, the limits exposed by failed cases and two or three questions that could justify thesis-scale work. Identify missing skills, data or hardware without claiming they have been obtained. The handoff should make it easy to extend the argument rather than restart an undocumented prototype.